

Handling Continuous Features

So far, we have only worked with **categorical features** (e.g., "Pointy" vs. "Floppy"). This lesson explains how decision trees can also split on **continuous features** (e.g., any number, like weight).

The Problem

How can a decision tree split a feature like `weight`, which can be any numerical value (e.g., 7.5, 9.1, 13.2)?

The Solution: Thresholding

Instead of splitting on discrete categories, the algorithm splits the data by finding an optimal **threshold**. The decision node's question becomes a binary (True/False) comparison.

- **Example:** Is `weight <= 9`?
 - If **True**, go to the left branch.
 - If **False** (meaning `weight > 9`), go to the right branch.
-

How to Find the Best Threshold?

The learning algorithm must determine the best possible threshold to use. It does this by testing many different values and calculating the **information gain** for each one.

The Process:

1. **Try a Threshold (e.g., `weight <= 8`):**
 - This splits the 10 examples into two groups:
 - **Left Branch:** 2 examples (2 Cats, 0 Dogs)
 - **Right Branch:** 8 examples (3 Cats, 5 Dogs)
 - The algorithm calculates the weighted average entropy and finds the **Information Gain = 0.24**.
2. **Try Another Threshold (e.g., `weight <= 9`):**
 - This splits the 10 examples into different groups:
 - **Left Branch:** 4 examples (4 Cats, 0 Dogs)
 - **Right Branch:** 6 examples (1 Cat, 5 Dogs)
 - This split is much "purer." The algorithm calculates the information gain and finds **Information Gain = 0.61**. This is much better!
3. **Try Another Threshold (e.g., `weight <= 13`):**

- This split results in **Information Gain = 0.40**.

The algorithm will test many possible thresholds to find the one that **maximizes information gain**. In this case, `weight <= 9` (with a gain of 0.61) is the best split for this feature.

The Full Splitting Algorithm

At any given node, the algorithm performs the following steps:

1. **For each categorical feature** (e.g., `Ear Shape`): Calculate its information gain.
 2. **For each continuous feature** (e.g., `weight`):
 - a. Find the **best possible threshold** by testing many different values and calculating the information gain for each.
 - b. The "score" for this feature is the *highest* information gain it can achieve (e.g., 0.61 for `weight`).
 3. **Final Decision:** Compare the best scores from all features. The algorithm splits on the feature that provided the **single highest information gain** overall.
 - (e.g., If `Ear Shape` gave 0.28 gain and `weight` gave 0.61, the algorithm would choose to split the node on `weight <= 9`).
-

Pro-Tip: How are thresholds chosen to be tested?

How does the algorithm test "many different values" efficiently? A common convention is:

1. **Sort** all examples at the node based on the continuous feature's value (e.g., sort all 10 animals by `weight`).
2. Test the **midpoint** value between each consecutive pair of examples.
3. (This way, if you have 10 examples, you only need to test 9 different thresholds to find the best possible split.)